

P5 Report

Scenario Justification

Scenario 1 - The waiting list for seats to a show

A queue is the most suitable data structure for a waiting list because it follows the FIFO, principle. The person who joins the waiting list first should have the most priority so that everyone would have fair allocation. Stack would be inappropriate because it follows LIFO, meaning most recently added the person would receive the seat first.

Scenario 2 - List of followers of a user on TipTop

Neither a queue nor a stack. Users do not genuinely need to be processed in LIFO or FIFO order. A stack or queue would impose an unnecessary ordering rule and make accessing a user inefficiently. A more suitable structure would be a array, list or another collection.

Scenario 3 - Browser History

A stack is suitable for browser history because the back button should return to the most recently visited page first. This follows the LIFO principle. Each newly visited page can be pushed onto the stack, and pressing back can pop the current page to return to the previous one. Queue would be unsuitable because FIFO would return to the oldest visited page first instead of the most recently visited page.

Evidence of Testing

/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P5/P5_task

1. Push a value onto the stack
2. Pop the top value off the stack
3. Peek at the top of the stack
4. Enqueue a value into the queue
5. Dequeue the front value from the queue
6. Peek at the front of the queue
7. See the current stack size
8. See the menu again
9. Quit

Enter command (1-9):

1

Enter the value to push:

1

Enter command (1-9):

1

Enter the value to push:

2

Enter command (1-9):

1

Enter the value to push:

3

Enter the value to push:

4

Enter command (1-9):

1

Enter the value to push:

5

Enter command (1-9):

1

Enter the value to push:

6

1. Push a value onto the stack
2. Pop the top value off the stack
3. Peek at the top of the stack
4. Enqueue a value into the queue
5. Dequeue the front value from the queue
6. Peek at the front of the queue
7. See the current stack size
8. See the menu again
9. Quit

Enter command (1-9):

7

Stack size: 6

Enter command (1-9):

3

Top of stack: 6

```
Enter command (1-9):  
2  
Popped: 6  
Enter command (1-9):  
2  
Popped: 5  
Enter command (1-9):  
2  
Popped: 4  
Enter command (1-9):  
2  
Popped: 3  
Enter command (1-9):  
2  
Popped: 2  
Enter command (1-9):  
2  
Popped: 1  
Enter command (1-9):  
2  
Popped: Error: Stack is empty
```

Enter command (1-9):

4

Enter the value to enqueue:

1

Enter command (1-9):

4

Enter the value to enqueue:

2

Enter command (1-9):

4

Enter the value to enqueue:

3

Enter command (1-9):

4

Enter the value to enqueue:

4

Enter command (1-9):

4

Enter the value to enqueue:

5

Enter command (1-9):

4

Enter the value to enqueue:

6

1. Push a value onto the stack
2. Pop the top value off the stack
3. Peek at the top of the stack
4. Enqueue a value into the queue
5. Dequeue the front value from the queue
6. Peek at the front of the queue
7. See the current stack size
8. See the menu again
9. Quit

Enter command (1-9):

6

Front of queue: 1

Enter command (1-9):

5

Dequeued: 1

Enter command (1-9):

5

Dequeued: 2

Enter command (1-9):

5

Dequeued: 3

Enter command (1-9):

5

Dequeued: 4

Enter command (1-9):

5

Dequeued: 5

Enter command (1-9):

5

Dequeued: 6

Enter command (1-9):

5

Dequeued: Error: Queue is empty